

Document technique final

Plateforme de gestion de cabinet d'audioprothèse — Solution DevOps hybride

Membre	Rôle principal	Périmètre
Zaafir Mougammadou Zaccaria	Infrastructure & déploiement	Terraform (AKS, ACR, PostgreSQL, réseau, budget), application FastAPI
Elyess Rjafellah	CI/CD & automatisation	GitHub Actions (deploy, backup, CI, sécurité), état Terraform distant, pipeline
Adame Nianghane	Observabilité & sécurité	Prometheus / Grafana / Loki / Alertmanager, DevSecOps (Trivy, CodeQL)
Anis Douadi	Kubernetes & on-premise	CIPPA Helm, HPA, NetworkPolicies, VM on-premise, MinIO chiffré, Ansible

SUP DE VINCI — Promotion 2025-2026

Code promo : **CODEPROMO** (à compléter)

1. Présentation de l'entreprise et de l'équipe

Le client. « AudioPro » est un cabinet d'audioprothèse qui assure le suivi de patients appareillés. Le cabinet doit gérer un référentiel de patients, leurs appareils auditifs et les rendez-vous de suivi, tout en garantissant la confidentialité des données de santé, la disponibilité du service et la maîtrise des coûts informatiques.

La mission. Concevoir et déployer, dans une démarche DevOps complète, une plateforme web fiable, sécurisée et supervisée, reproductible et automatisée de bout en bout.

L'équipe projet. Quatre étudiants de Mastère DevOps, avec une répartition des rôles complémentaire couvrant l'ensemble de la chaîne (développement, infrastructure, automatisation, observabilité, sécurité, orchestration, reprise d'activité) :

Membre	Rôle principal	Périmètre
Zaafir Mougammadou Zaccaria	Infrastructure & déploiement	Terraform (AKS, ACR, PostgreSQL, réseau, budget), application FastAPI
Elyess Rjafellah	CI/CD & automatisation	GitHub Actions (deploy, backup, CI, sécurité), état Terraform distant, pipeline
Adame Nianghane	Observabilité & sécurité	Prometheus / Grafana / Loki / Alertmanager, DevSecOps (Trivy, CodeQL)
Anis Douadi	Kubernetes & on-premise	CI/CD Helm, HPA, NetworkPolicies, VM on-premise, MinIO chiffré, Ansible

2. Analyse de la problématique et solution proposée

2.1 Problématique

- **Données sensibles** : informations de santé soumises au RGPD (confidentialité, traçabilité).
- **Disponibilité** : le service doit rester accessible et se rétablir seul en cas d'incident.
- **Sécurité** : chaîne logicielle et exécution durcies, secrets jamais exposés.
- **Scalabilité** : absorber des pics de charge sans intervention manuelle.
- **Coûts** : rester dans le budget d'un compte Azure for Students (85 \$).
- **Reproductibilité** : pouvoir reconstruire l'ensemble à l'identique et automatiquement.

2.2 Solution proposée

Une application web conteneurisée (API FastAPI, frontend React, base PostgreSQL) déployée sur un cluster **Kubernetes managé (Azure AKS)**, encadrée par une chaîne DevOps complète : infrastructure as code (Terraform), intégration et déploiement continu (GitHub Actions), supervision (Prometheus / Grafana / Loki), sécurité intégrée (DevSecOps) et plan de reprise d'activité. L'architecture est **hybride** : le cloud héberge l'application, un site **on-premise simulé** (réseau séparé) héberge le stockage objet chiffré (MinIO) des sauvegardes.

2.3 Architecture (vue d'ensemble)

Développeur → **GitHub** → **GitHub Actions** (lint, tests, scans de sécurité, build, déploiement) → images poussées sur **Azure Container Registry** → déploiement **Helm** sur **AKS**. Les utilisateurs accèdent à l'application via un **Ingress NGINX** ; le backend communique avec **PostgreSQL Flexible Server** (TLS). La supervision (Prometheus/Grafana/Loki/Alertmanager) et la sécurité (RBAC, NetworkPolicies, secrets) sont intégrées au cluster. Les sauvegardes chiffrées sont répliquées vers une **VM on-premise** (MinIO), pilotée par Ansible.

3. Gestion des coûts — FinOps (spécifique M2)

Le compte Azure for Students est limité à **85 \$**, sans carte bancaire : la maîtrise des coûts est une contrainte de conception.

Ressource	Dimensionnement	Coût indicatif 24/7
AKS — plan de contrôle	Free tier	0 \$
AKS — nœuds	2 × Standard_B2s_v2	~60-70 \$/mois
PostgreSQL Flexible	B_Standard_B1ms	~13 \$/mois
Azure Container Registry	Basic	~5 \$/mois
VM on-premise (MinIO)	Standard_B2s_v2	~30 \$/mois
Équilibreurs / IP publiques	Standard	~6-9 \$/mois

3.1 Leviers d'optimisation

- Plan de contrôle AKS gratuit, nœuds **burstable** (série B), SKU minimaux partout.
- **Loki** à la place d'ELK (indexation des labels uniquement) : coût d'exploitation très réduit.
- Secrets dans GitHub Actions plutôt qu'un Key Vault hébergé : moins de ressources facturées.
- Rétention courte (métriques 3 j, sauvegardes 7 j) et autoscaling **borné** (HPA max 2).
- Grafana exposé via l'Ingress existant (pas d'IP publique supplémentaire).

3.2 Garde-fous et extinction

- **Budget Azure** avec alertes e-mail à 50 / 80 / 100 % du plafond mensuel.
- Étiquetage (tags) de toutes les ressources pour le suivi analytique (Cost Management).
- **Teardown en un clic** (workflow Deploy → action *destroy*) : facturation ramenée à ~0 \$ hors période de démonstration ; reconstruction complète en ~12-14 min.

4. Organisation, planification et méthodologie

4.1 Méthodologie

Démarche **agile (Scrum/Kanban)** en sprints de deux semaines, priorisation MoSCoW. Outils professionnels : Git/GitHub (versioning + revues de code via Pull Requests), GitHub Actions (CI/CD), Terraform et Ansible (gestion de configuration), Helm (packaging Kubernetes).

4.2 Planification

Jalon	Échéance	Livrable
M1 — Cadrage	Kick-off +1 mois	Backlog, architecture, choix FinOps
M2 — MVP applicatif	+3 mois	Application conteneurisée + tests
M3 — Déploiement cloud	+4,5 mois	Infra Terraform + CI/CD + monitoring
M4 — Vidéo MVP	+6 mois	Démonstration en production
M5 — Rendu technique	+6 mois	Dépôt + documentation + analyse

4.3 Contributions individuelles

Le travail a été réparti de façon équilibrée ; chaque membre est responsable d'un domaine de la chaîne DevOps et a contribué au dépôt versionné (commits, PR, revues). La traçabilité est visible dans l'historique Git et l'onglet *Insights* → *Contributors* de GitHub.

Membre	Rôle principal	Périmètre
Zaafir Mougammadou Zaccaria	Infrastructure & déploiement	Terraform (AKS, ACR, PostgreSQL, réseau, budget), application FastAPI
Elyess Rjafellah	CI/CD & automatisation	GitHub Actions (deploy, backup, CI, sécurité), état Terraform distant, pip
Adame Nianghane	Observabilité & sécurité	Prometheus / Grafana / Loki / Alertmanager, DevSecOps (Trivy, CodeQL)
Anis Douadi	Kubernetes & on-premise CI/CD	CRP Helm, HPA, NetworkPolicies, VM on-premise, MinIO chiffré, Ansible

5. Présentation de la solution technique

Domaine	Technologies
Backend	Python 3.11, FastAPI, SQLAlchemy, Pydantic
Frontend	React 18, Vite (servi par nginx non-root)
Base de données	PostgreSQL 16 (Azure Flexible Server, TLS)
Conteneurs / Orchestration	Docker (images non-root), Kubernetes (AKS), Helm
IaC / Configuration	Terraform (azurerem), Ansible
CI/CD	GitHub Actions (deploy, backup, CI, sécurité)
Observabilité	Prometheus, Grafana, Loki, Alertmanager
Sécurité	Trivy, CodeQL, Gitleaks, RBAC, NetworkPolicies, TLS
On-premise / PRA	VM Linux, MinIO chiffré (SSE), sauvegarde/restauration Ansible

5.1 CI/CD — déploiement 100 % automatisé

Un **push sur la branche principale** (ou un déclenchement manuel) provisionne l'infrastructure (Terraform), construit et scanne les images, puis déploie l'application via Helm — sans aucune étape manuelle. Étapes : connexion Azure → état Terraform → *apply* → build/push ACR → scan Trivy (bloquant si vulnérabilité critique) → installation Ingress + monitoring → déploiement de l'app → configuration MinIO (Ansible). Le pipeline est **auto-réparant** (gestion des verrous d'état et des releases bloquées).

5.2 Orchestration Kubernetes

- Chart **Helm** paramétrable (Deployments, Services, Ingress, Secret, HPA).
- **HPA** (autoscaling CPU), sondes *liveness/readiness*, auto-réparation des pods.
- **RBAC**, **NetworkPolicies** et *securityContext* durci (non-root, capabilities drop).

5.3 Observabilité

Métriques exposées par l'API (/metrics) et collectées par Prometheus ; tableaux de bord Grafana (débit, latence p95, taux d'erreurs, disponibilité) ; logs centralisés via Loki ; alertes routées par Alertmanager vers Slack/Teams.

5.4 Sécurité (DevSecOps)

Scans automatiques à chaque exécution : Trivy (dépendances, IaC, images), CodeQL (analyse statique), Gitleaks (fuite de secrets). Chiffrement TLS de la base, secrets jamais versionnés, hébergement en région européenne (RGPD).

5.5 PRA / PCA — testé et prouvé

Sauvegarde chiffrée de la base vers MinIO (on-premise) et restauration automatisée via Ansible. Le cycle complet a été **validé en conditions réelles** : création d'une donnée témoin → sauvegarde → suppression (perte simulée) → restauration → donnée récupérée. La continuité de service (PCA) est assurée par l'auto-réparation Kubernetes.

5.6 Validation

La solution a été déployée et testée de bout en bout : application et API fonctionnelles, base joignable, supervision active, résilience vérifiée (recréation automatique des pods), scans de sécurité au vert, et PRA prouvé. L'ensemble est reproductible automatiquement.